Universidad de los Andes Facultad de Ingeniería	**Ingeniería de Sistemas y Computación** **ISIS-1611: Inteligencia artificial** Semestre: 2026-10 Créditos: 3	Accredited ABET Engineering Accreditation Commission

Taller 2: Restricciones, Adversarios y Drones

Las comunidades indígenas del Amazonas colombiano han sido aisladas por una inundación estacional severa que destruyó los únicos caminos de acceso terrestre. La Unidad de Gestión del Riesgo y Desastres ha desplegado una flota de drones para llevar suministros médicos críticos (vacunas, sueros antiofídicos y material quirúrgico) a las comunidades afectadas, donde los tiempos de respuesta pueden ser la diferencia entre la vida y la muerte.

La selva plantea desafíos operacionales únicos: el dosel arbóreo genera zonas de interferencia electromagnética, las tormentas tropicales crean corredores de viento peligrosos, y la vegetación densa obstruye ciertas rutas de vuelo. Pero el mayor riesgo son los cazadores furtivos que operan en la región y que, ante el valor de los componentes electrónicos del dron, intentan interceptarlo usando inhibidores de señal RF cuando logran posicionarse en la misma zona que el dron.

El desafío ahora tiene dos componentes:

1) La asignación de rutas seguras para cada uno de los drones distribuyendo los destinos entre ellos mientras se respetan restricciones estrictas de batería, capacidad de carga y ventanas de tiempo.
2) La necesidad de los drones de evitar a los cazadores furtivos que intentan vender sus partes

Su responsabilidad es diseñar e implementar, primero, un solucionador de Problemas de Satisfacción de Restricciones (CSP) que permita planificar las misiones de forma segura y eficiente; y segundo, agentes de búsqueda adversaria que permitan al dron navegar en vuelo evitando a los cazadores, comparando diferentes estrategias y desarrollando funciones de evaluación sofisticadas.

Objetivos de aprendizaje

1. Modelar un problema de asignación como un CSP, identificando correctamente variables, dominios y restricciones binarias y n-arias.
2. Implementar búsqueda por backtracking con propagación de restricciones (AC-3, Forward Checking) y analizar cómo la consistencia reduce el espacio de búsqueda.
3. Implementar Minimax para un agente que compite contra adversarios en un entorno discreto.
4. Implementar Expectimax para modelar adversarios que actúan de forma probabilística.
5. Diseñar funciones de evaluación no triviales para estados intermedios de un juego adversario.
6. Analizar críticamente las propiedades de completitud, optimalidad, complejidad espacial y temporal de cada enfoque, y comparar experimentalmente su desempeño.

Escenario del Problema

El dron de la Unidad de Gestión del Riesgo y Desastres cuenta con las siguientes características:

- Puede desplazarse únicamente en 4 direcciones: arriba, abajo, izquierda y derecha.
- No puede atravesar obstáculos impasables ni salirse de los límites del mapa.

- Debe entregar suministros médicos en todos los puntos de entrega asignados.
- Cuenta con una capacidad de batería limitada. El costo de desplazamiento varía según el tipo de terreno:

Símbolo	Tipo de Terreno	Costo por Paso	Descripción
.	Espacio libre	1	Zona despejada. Vuelo normal sin interferencias.
~	Zona de niebla	2	Niebla densa o interferencia atmosférica leve. Visibilidad reducida y mayor consumo de batería.
^	Terreno montañoso	3	Zona de altitud elevada. El dron debe ganar altura. Alto consumo de energía.
*	Tormenta eléctrica	5	Zona de alta peligrosidad eléctrica. Extremadamente peligrosa. Costo de batería máximo. Evitar si es posible.
%	Obstáculo impasable	∞	Montaña u obstáculo rígido. El dron no puede ocupar esta celda bajo ninguna circunstancia.
D	Dron (posición inicial)	N/A	Punto de partida del dron al inicio de la simulación. Puede haber múltiples (D1, D2…)
E	Punto de entrega	N/A	Destino donde se deben depositar suministros. Puede haber múltiples (E1, E2…).
C	Cazador	N/A	Agente adversario que intenta interceptar al dron. Puede haber múltiples (C1, C2…).

Los cazadores (C) portan inhibidores de RF que solo funcionan cuando el cazador comparte exactamente la misma casilla del grid que el dron. La captura ocurre cuando ambos ocupan la misma posición en el mismo turno. Los cazadores solo pueden caminar por espacio libre, por lo cual el dron puede ganar una ventaja para escapar incurriendo en un mayor costo de batería.

El juego adversario es por turnos. En cada turno completo: (1) el dron elige una acción, y (2) cada cazador elige una acción. El dron pierde si un cazador llega a su casilla. El dron gana si entrega todos los suministros médicos.

Estructura del Proyecto

Usted recibe un esqueleto base funcional que incluye toda la infraestructura de simulación y visualización. Su responsabilidad es completar los algoritmos en los archivos designados:

- `algorithms/csp.py`: Implementación del solucionador CSP: backtracking, AC-3, Forward Checking, MRV y LCV.
- `algorithms/adversarial.py`: Implementación de los agentes de búsqueda adversaria: Minimax, Alfa-Beta y Expectimax.
- `algorithms/evaluation.py`: Función de evaluación heurística para estados no terminales del juego adversario.

Aunque no debe modificar los demás archivos, puede consultarlos para entender detalles de su implementación:

- Módulo `algorithms/`:
  - `problems_csp.py`: Define `DroneAssignmentCSP`: variables, dominios, restricciones e implementa `is_consistent`() y `assign`().
  - `utils.py`: Proporciona algoritmos útiles para ayudar en la solución de las tareas.

- Módulo `world/`:
  - `game_state.py`: Guarda el estado actual del juego.
  - `rules.py`: Implementa la lógica del agente cazador greedy y aleatorio.
  - `layout.py`: Carga los archivos con formato.lay e interpreta todos los símbolos.
  - `game.py`: Coordina el estado del juego incluyendo posición del dron, posiciones de cazadores, entregas pendientes y mapa del juego.
- Módulo `view/`:
  - Contiene los módulos de visualización para interfaz gráfica y para modo texto.
- Carpetas de `layouts/`:
  - `csp/`: Mapas para el modo CSP. No contienen cazadores, pero definen restricciones.
  - `adversarial`: Mapas para el modo adversario. Contienen agentes cazadores.

Ejecución

El programa tiene dos modos de operación según la fase del taller.

Para el modo **CSP**:

```
python main.py -m csp -a ALGORITMO_CSP -l LAYOUT [OPCIONES]
```

Para el modo **adversario**:

```
python main.py -m adversarial -a AGENTE -l LAYOUT [OPCIONES]
```

Las opciones disponibles se describen a continuación:

Opción	Descripción	Default	Ejemplo
`-m`	Modo de operación: `csp` para planificación o `adversarial` para navegación.	Ninguno	`-m adversarial`
`-a`	Según modo de operación: - Algoritmo CSP: backtracking, backtrackingFC, backtrackingAC3, backtrackingMRV_LCV. - Agente adversario: MinimaxAgent, AlphaBetaAgent, ExpectimaxAgent.	Ninguno	`-a backtracking`
`-d`	Profundidad de búsqueda en el árbol de juego.	`1`	`-d 3`
`-p`	Probabilidad de que el cazador ejecute una acción aleatoria (un cazador greedy es $p = 0$ para $0 \leq p \leq 1$)	`0`	`-p 0.2`
`-l`	Archivo de mapa a cargar (sin extensión .lay).	Ninguno	`-l small_hunt`
`-n`	Número de partidas a ejecutar en secuencia (para estadísticas).	`1`	`-n 20`
`-t`	Activa modo de visualización por texto. Útil si no dispone de interfaz gráfica.	Desactivado	`-t`
`-q`	Genera salida mínima en consola, sin gráficos. Ideal para múltiples ejecuciones.	Desactivado	`-q`
`-x`	Tiempo de espera en segundos entre fotogramas. Valor negativo: avance manual (por teclado)	`0.1`	`-x 0.3`
`-z`	Ajusta el tamaño de la ventana gráfica (si aplica). Un valor de 2.0 duplica el tamaño, 0.5 lo reduce a la mitad.	1.0	`-z 2.0`
`--help`	Muestra la ayuda con todas las opciones disponibles.	—	`--help`

Puede verificar que el proyecto se ejecuta correctamente con el siguiente ejemplo de referencia:

```
python main.py -m adversarial -a RandomAgent -l tiny_hunt -x 0.5 -z 2.0
```

Tip: En el archivo `README.md` del código base podrá encontrar los comandos que puede ejecutar para probar cada uno de los puntos de este taller. Puede elegir cuáles desea utilizar para su análisis.

Nota: El proyecto utiliza la librería tkinter para mostrar una interfaz gráfica. Si en su computadora tkinter no está instalado o genera errores, use el modo texto (-t) o el modo silencioso (-q).

Punto 1: Asignación de Múltiples Drones a Múltiples Destinos (35%)

La Unidad de Gestión del Riesgo y Desastres opera una flota de n drones que deben cubrir m puntos de entrega simultáneamente. Cada dron tiene una capacidad de carga máxima (en kg) y un alcance de batería (en unidades de costo de ruta). Cada punto de entrega tiene un peso de suministros y una ventana de tiempo $[t_{early}, t_{late}]$ dentro de la cual debe recibir la entrega.

El CSP de asignación se formula así: las variables A_i (para cada punto de entrega i) representan el dron asignado a esa entrega; el dominio de cada A_i es el conjunto de drones disponibles; las restricciones son (1) capacidad: la suma de pesos de entregas asignadas a un dron no supera su capacidad, (2) alcance: la ruta mínima a todos sus destinos no supera su batería, (3) ventanas de tiempo: el dron debe poder llegar a la entrega i dentro de $[t_{early_i}, t_{late_i}]$, y (4) secuencialidad: un dron solo puede atender sus entregas en secuencia, no en paralelo. Dentro de su asignación, los drones priorizan entregar los suministros al punto de entrega con menor t_{early_i} y pueden esperar. Los drones deben regresar a su punto de partida después de realizar cada una de sus entregas.

Implemente en `algorithms/csp.py` las siguientes funciones:

a) `backtracking_search`: Backtracking básico sin ninguna optimización. Retorna un diccionario `{variable: valor}` con la asignación completa, o `None` si no existe solución.
b) `backtracking_fc`: Forward Checking. Al asignar una variable, propaga las restricciones hacia las variables no asignadas y elimina valores inconsistentes de sus dominios.
c) `backtracking_ac3`: Ejecuta AC-3 antes de iniciar la búsqueda para reducir dominios y mantiene la consistencia de arco tras cada asignación.
d) **(Bono 5%)** `backtracking_mrv_lcv`: Backtracking con Forward Checking que usa heurísticas MRV (Minimum Remaining Values) para seleccionar variables y LCV (Least Constraining Value) para ordenar valores.

Pruebe sus implementaciones con los layouts de `layouts/csp`. Compare los algoritmos en términos de número de asignaciones intentadas y tiempo de ejecución. Valide sus propias hipótesis cambiando las opciones de ejecución y creando layouts personalizados. Este proceso de experimentación puede enriquecer su entendimiento y análisis.

Tip: La clase `DroneAssignmentCSP` en `algorithms/problems_csp.py` ya está implementada: define variables, dominios, restricciones y el método `is_consistent()`. Usted solo debe implementar los algoritmos de solución de CSP en `algorithms/csp.py`.

Punto 2: Dron vs. Cazadores (30%)

El dron ya está en vuelo y los sistemas de detección han identificado uno o más cazadores. Los cazadores actúan de forma óptima y determinista: siempre eligen la acción que mayor riesgo causa

al dron (en este caso, minimizar la distancia Manhattan). Usted modelará esta situación como un juego de suma cero y lo resolverá con Minimax y su variante con Poda Alfa-Beta.

El juego adversario tiene la siguiente definición: el dron es el agente MAX; cada cazador es un agente MIN independiente. En cada turno completo el dron mueve primero, luego cada cazador mueve en orden. El estado es terminal si un cazador comparte casilla con el dron (derrota, utilidad −1000) o si el dron ha visitado todos los puntos de entrega (victoria, utilidad +1000).

Implemente en `algorithms/adversarial.py` las siguientes clase:

a) `MinimaxAgent`: Retorna la acción óptima del dron dado el estado actual. Con múltiples cazadores, el árbol debe tener una capa MIN por cada cazador antes de retornar a MAX.
b) **(Bono 5%)** `AlphaBetaAgent`: Extiende MinimaxAgent con Poda Alfa-Beta. Verifique que el agente produce las mismas acciones que `MinimaxAgent` explorando menos nodos

Implemente en `algorithms/evaluation.py`:

c) `evaluation_function(state)`: Función que evalúa un estado no terminal del juego asignando un puntaje real. Esta función es clave para que su agente aprenda a reconocer un estado deseable. Si el agente no actúa como usted espera, considere mejorar la función.

Pruebe sus agentes en los layouts de `layouts/adversarial`. Describa cualitativamente el desempeño de sus agentes y compare cuantitativamente su complejidad computacional.

Punto 3: Cazadores Estocásticos (20%)

En escenarios reales, los cazadores no siempre actúan de forma perfectamente racional. Análisis de campo revelan que los cazadores siguen una política mixta: en ocasiones se dirigen hacia la posición actual del dron, y en otras eligen una dirección aleatoria. Expectimax utiliza nodos de azar que calculan el valor esperado bajo la distribución de acciones del cazador.

Implemente en `algorithms/adversarial.py`:

a) `ExpectimaxAgent`: Los nodos del dron siguen siendo MAX. Los nodos de los cazadores son nodos de azar que calculan el promedio ponderado de los valores de sus sucesores. Experimente con varios valores de p, la probabilidad de que un cazador actúe al azar.

Debido a la naturaleza aleatoria de este problema, se recomienda utilizar el argumento `-n` para realizar varias ejecuciones del programa y obtener métricas de rendimiento más confiables.

Punto 4: Reflexión sobre el trabajo realizado (15%)

La Unidad de Gestión del Riesgo y Desastres desea mejorar el funcionamiento de su flota de drones para incrementar las probabilidades de éxito de futuras misiones. Con este fin, le ha solicitado un análisis a partir de su experiencia diseñando algoritmos de CSP y búsqueda adversaria. Conteste las siguientes preguntas en un documento PDF:

a) (6%) Según los algoritmos CSP que haya implementado, indique claramente:
   - Complejidad en espacio (Notación O) en función del número de variables n, tamaño máximo del dominio d y número de restricciones c.
   - Complejidad en tiempo (Notación O).
   - ¿Cómo se reduce el espacio de búsqueda comparado con una búsqueda exhaustiva simple? Compare cuantitativamente el número de asignaciones intentadas con ejemplos extraídos de sus experimentos.
b) (6%) Respecto a la búsqueda adversaria, responda:

- Complejidad en tiempo y espacio de Minimax en función de la profundidad d y el factor de ramificación b.
- ¿Por qué Minimax puede ser pesimista en entornos con cazadores no óptimos?
- Describa y justifique su función de evaluación: ¿qué características del estado usa, qué pesos asignó y por qué?
- Compare las tasas de victoria de los agentes en diversos layouts.
- ¿Cuántos nodos adicionales poda Alfa-Beta en los mejores y peores casos?

Se espera que en su análisis compare cualitativamente el desempeño de los distintos algoritmos en los layouts que encuentre de interés, así como que compare cuantitativamente métricas de complejidad computacional, tasa de victorias y tiempo de ejecución.

c) (3%) Escriba una reflexión concisa sobre la co-construcción de la solución final con apoyo de la IA, indicando, por ejemplo, qué aprendizajes obtuvo al ver las correcciones sugeridas, la dificultad para crear el *prompt* adecuado, si la solución propuesta fue fácil de entender (¿usted la podría explicar en una sustentación?). Si no hizo ninguna iteración con IA, indique por qué no le pareció necesario. Este punto busca que usted se tome el tiempo de reflexionar sobre el uso de la IA en tareas de programación.

Política de Uso de IA Generativa y presentación en el trabajo a entregar

Para este taller se espera que usted desarrolle una primera versión de la solución de forma completamente autónoma, usando su propio criterio, conocimiento y creatividad antes de recurrir a herramientas de IA. La IA puede emplearse después para mejoras puntuales, refactorización, comentarios de calidad o apoyo en la corrección de errores, pero nunca como sustituto del esfuerzo personal ni como generador principal del código. Use estas herramientas con criterio profesional, asumiendo responsabilidad sobre su proceso de aprendizaje y entendiendo que lo que más valor tiene aquí es el camino que usted recorre para llegar a la solución, no solo el resultado final.

Todos los prompts y las versiones de código generadas con apoyo de IA deben quedar registrados dentro de los archivos del proyecto. Si utiliza IA para refactorizar u optimizar su solución, incluya en el archivo: (1) la versión inicial del código, (2) los prompts que utilizó para refinarla y (3) la versión final del código. Al finalizar, deje como código activo únicamente la versión final y conserve la versión inicial y todos los prompts en forma de comentarios dentro del mismo archivo.

Entrega del trabajo

- Guarde todo su trabajo (incluyendo el documento PDF) en una carpeta comprimida (.zip).
- Suba el trabajo al espacio correspondiente en Bloque Neón a más tardar la fecha y hora indicadas (un solo miembro del grupo debe subir el trabajo).
- Recuerde incluir los nombres y códigos de todos los integrantes de su grupo.

Nota: Al enviar su solución, usted declara que el código entregado en la primera versión de cada punto es de su autoría y que las versiones que utilizan IA fueron la respuesta a los prompts incluidos, que comprende el funcionamiento en todas las versiones y que acepta que este material pueda ser revisado, ejecutado y evaluado por el equipo docente para efectos académicos y de verificación de originalidad.